

《人工智能基础 A》实验报告 4

LSTM 时序预测与股票分析

学号

姓名: 董卓然

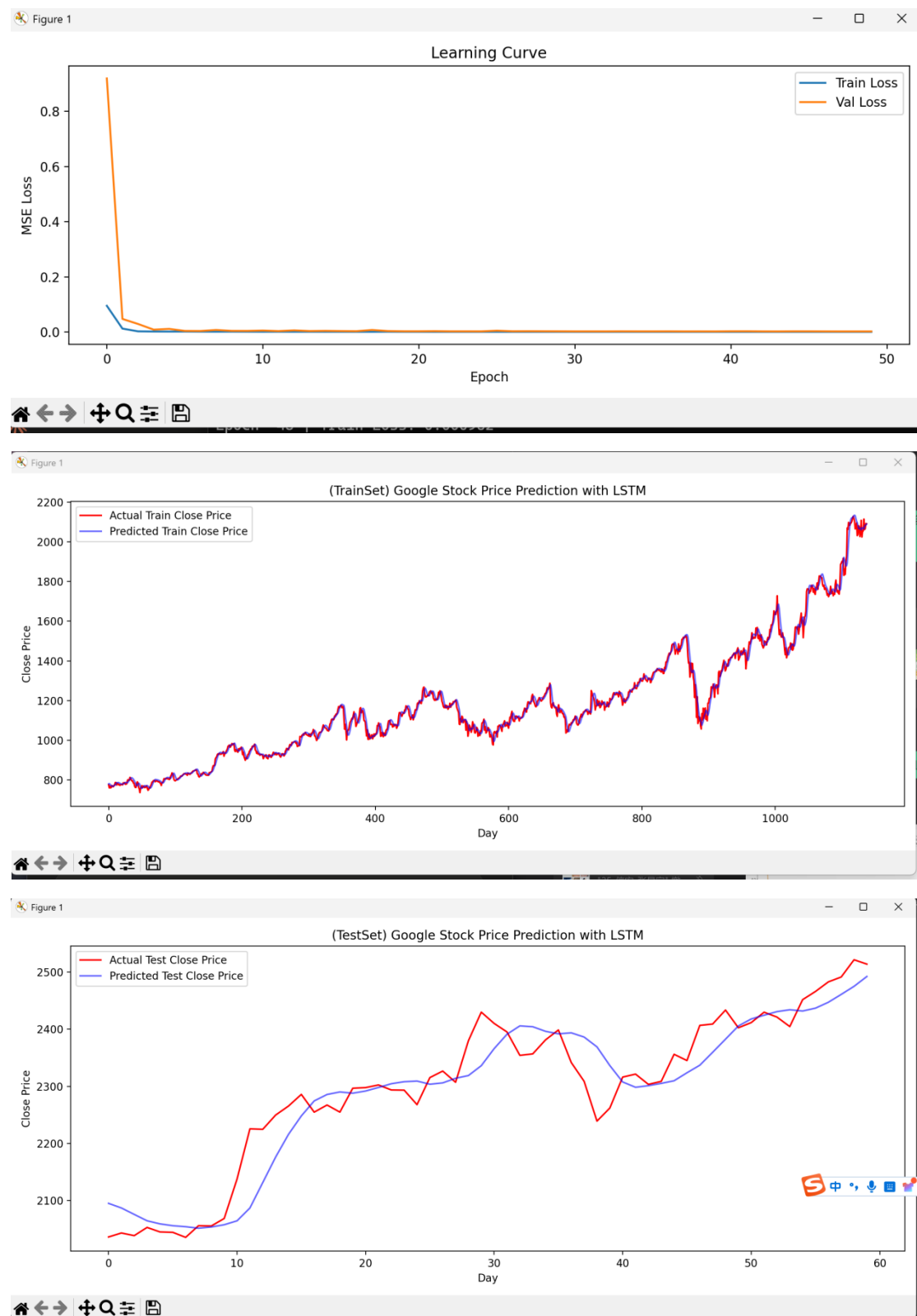
一、实验报告（实验文档中只包含此部分即可，将截图放在此处）

报告内容应完整覆盖实验要求，格式规范、排版整洁【占 10 分】。

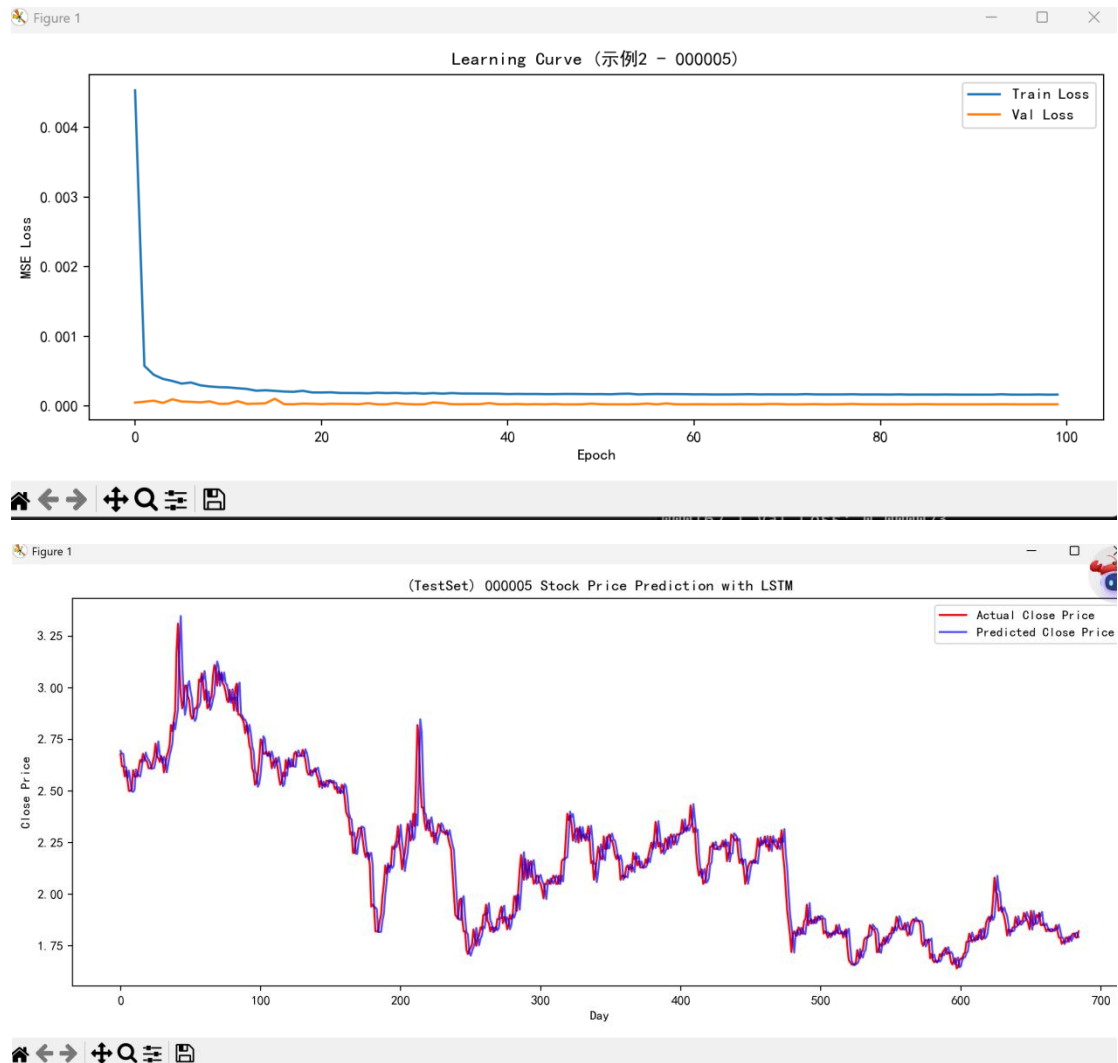
目标 1: 完成并完整运行两个示例【10】

```
Epoch 44 | Train Loss: 0.000958
           Val   Loss: 0.002660
[保存] Epoch 44 最佳模型已保存
Epoch 45 | Train Loss: 0.000954
           Val   Loss: 0.002999
Epoch 46 | Train Loss: 0.000955
           Val   Loss: 0.002975
Epoch 47 | Train Loss: 0.000974
           Val   Loss: 0.002762
Epoch 48 | Train Loss: 0.000952
           Val   Loss: 0.002632
[保存] Epoch 48 最佳模型已保存
Epoch 49 | Train Loss: 0.000961
           Val   Loss: 0.002676
Epoch 50 | Train Loss: 0.000956
           Val   Loss: 0.002631
[保存] Epoch 50 最佳模型已保存

===== 评估指标 =====
Train MAE: 18.9652
Test  MAE: 34.1355
```



【示例 1 运行结果截图】



【示例 2 运行结果截图】

1. 目标 2：在示例基础上（任选一个示例），采用其他股票数据集，自定义任务（如涨跌平分类、回归），并取得较好表现。【40】

1.1. 自定义任务说明

本任务选用 515180（华宝中证医疗 ETF）股票数据集，时间范围为 2019 年 12 月至 2026 年 5 月，共 1542 条日线数据。自定义任务为收盘价回归预测：以过去 30 个交易日的收盘价作为输入特征，预测下一交易日的收盘价。数据采用 Min-Max 标准化（归一化至 $[-1, 1]$ 区间），按 85%/15% 划分训练集和测试集。

1.2. 模型结构

本模型在示例 1 基础上进行改进，增加了 Dropout 层以防止过拟合。具体结构为：双层 LSTM（隐藏层维度分别为 50 和 64）后接两层全连接层，每层 LSTM 后添加 Dropout ($p=0.2$)。输入为形状 (batch, 30, 1) 的时序张量，输出为标量预测值。

```

PS C:\Users\lenovo\Desktop\实验 4> python target2_classification.py
使用设备: cpu
数据量: 1542 条, 时间范围: 2019-12-23 ~ 2026-05-08
训练集: 1285 条, 测试集: 227 条

模型结构:
ImprovedLSTM(
  (lstm1): LSTM(1, 50, batch_first=True)
  (drop1): Dropout(p=0.2, inplace=False)
  (lstm2): LSTM(50, 64, batch_first=True)
  (drop2): Dropout(p=0.2, inplace=False)
  (fc1): Linear(in_features=64, out_features=32, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=32, out_features=16, bias=True)
  (fc3): Linear(in_features=16, out_features=1, bias=True)
)
Epoch 1 | Train Loss: 0.116578 | Val Loss: 0.128573
  [保存] 最佳模型 Val Loss=0.128573
Epoch 2 | Train Loss: 0.027116 | Val Loss: 0.058185
  [保存] 最佳模型 Val Loss=0.058185
Epoch 3 | Train Loss: 0.012970 | Val Loss: 0.012113
  [保存] 最佳模型 Val Loss=0.012113
Epoch 4 | Train Loss: 0.010310 | Val Loss: 0.008515
  [保存] 最佳模型 Val Loss=0.008515
Epoch 5 | Train Loss: 0.009170 | Val Loss: 0.007723

```

```

PS C:\Users\lenovo\Desktop\实验 4> python target2_classification.py
使用设备: cpu
数据量: 1542 条, 时间范围: 2019-12-23 ~ 2026-05-08
训练集: 1285 条, 测试集: 227 条

模型结构:
ImprovedLSTM(
  (lstm1): LSTM(1, 50, batch_first=True)
  (drop1): Dropout(p=0.2, inplace=False)
  (lstm2): LSTM(50, 64, batch_first=True)
  (drop2): Dropout(p=0.2, inplace=False)
  (fc1): Linear(in_features=64, out_features=32, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=32, out_features=16, bias=True)
  (fc3): Linear(in_features=16, out_features=1, bias=True)
)
Epoch 1 | Train Loss: 0.116578 | Val Loss: 0.128573
  [保存] 最佳模型 Val Loss=0.128573
Epoch 2 | Train Loss: 0.027116 | Val Loss: 0.058185
  [保存] 最佳模型 Val Loss=0.058185
Epoch 3 | Train Loss: 0.012970 | Val Loss: 0.012113
  [保存] 最佳模型 Val Loss=0.012113
Epoch 4 | Train Loss: 0.010310 | Val Loss: 0.008515
  [保存] 最佳模型 Val Loss=0.008515
Epoch 5 | Train Loss: 0.009170 | Val Loss: 0.007723

```

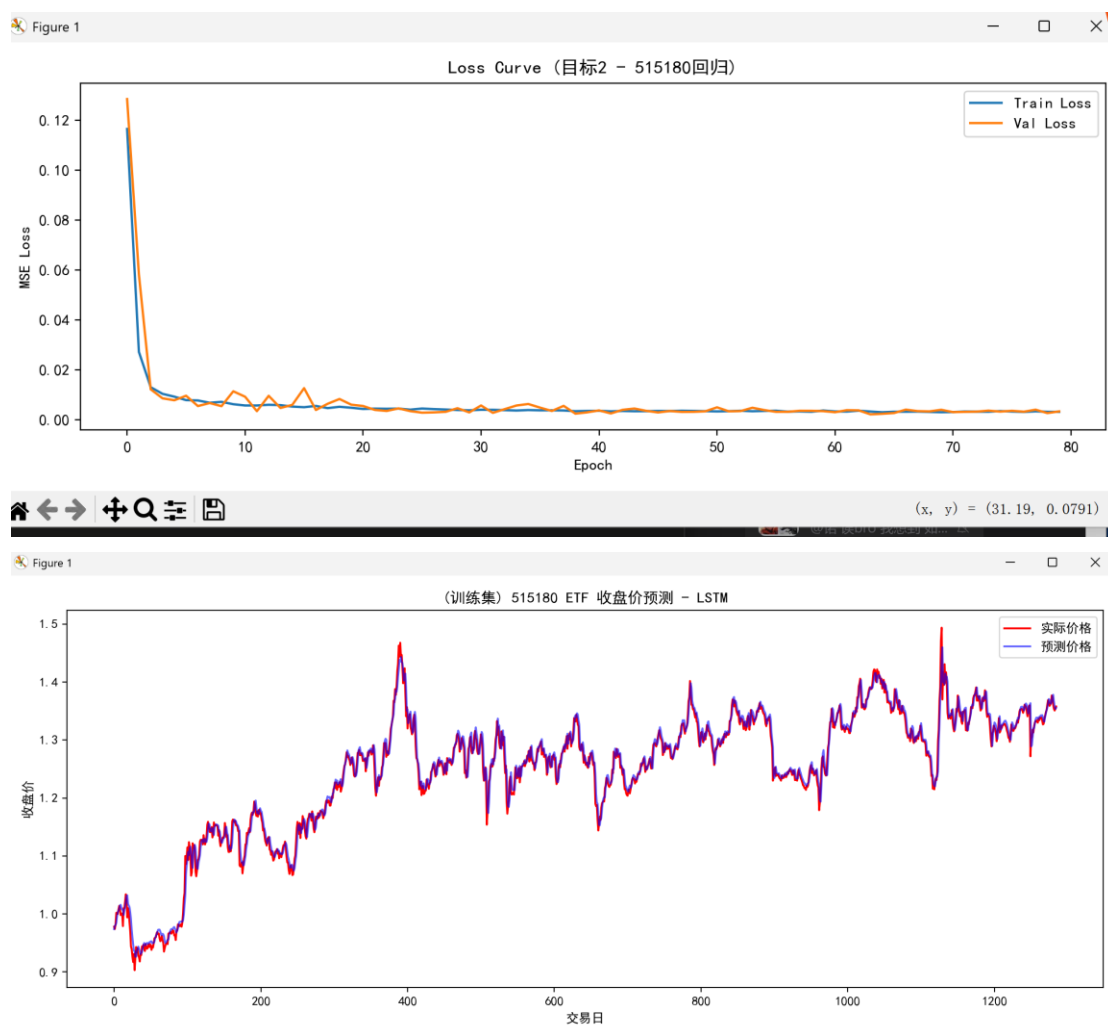
1.3. 损失函数

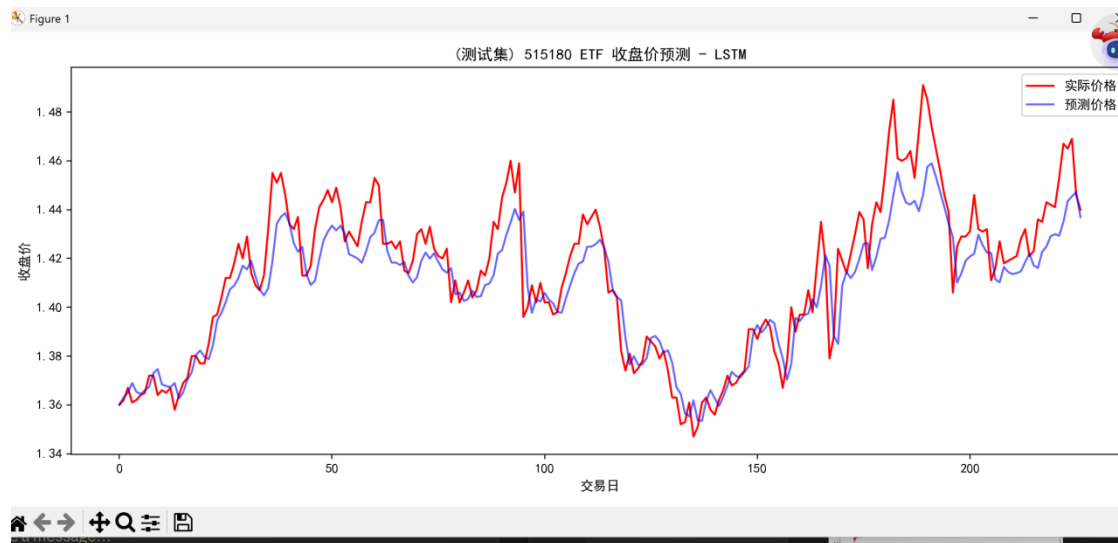
本任务为回归预测任务，损失函数采用均方误差损失（**MSELoss**），公式为：

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

优化器使用 AdamW（学习率 $1e-3$ ，权重衰减 $1e-4$ ），学习率调度策略为 StepLR（每 20 个 epoch 衰减为原来的 0.5 倍）。训练过程中对梯度进行裁剪（ $\text{max_norm}=1.0$ ）以防止梯度爆炸。

1.4 训练和验证结果





模型训练共 80 个 epoch，损失曲线显示训练集与验证集损失均快速下降并趋于稳定，未出现明显过拟合现象。预测曲线与真实价格走势高度吻合，能够有效捕捉价格的趋势变化。

2. 目标 3: 目标 2 的基础上,采用滚动回测进行训练和评估。

【40】

2.1. 滚动回测策略说明

1. 本实验采用前进式滚动回测 (Walk-Forward Backtesting) 策略，具体实现如下：设定固定训练窗口长度为 400 个交易日，验证窗口（步长）为 60 个交易日。每轮回测中，训练窗口向前滚动 60 天，在新的训练窗口上从头训练模型，并在紧随其后的 60 天验证窗口上评估预测效果。数据归一化仅使用当前训练窗口的统计量 (MinMaxScaler)，不使用验证窗口的数据，从而严格避免未来信息泄露。共进行 18 轮回测。

数据量：1542 条，时间范围：2019-12-23 ~ 2026-05-08

总数据量：1542 | 训练窗口：400 | 步长：60

预计轮次：18

轮次	1	训练:[0, 400)	验证:[400, 460)	样本: 60	MAE: 0.0595
轮次	2	训练:[60, 460)	验证:[460, 520)	样本: 60	MAE: 0.0156
轮次	3	训练:[120, 520)	验证:[520, 580)	样本: 60	MAE: 0.0268
轮次	4	训练:[180, 580)	验证:[580, 640)	样本: 60	MAE: 0.0128
轮次	5	训练:[240, 640)	验证:[640, 700)	样本: 60	MAE: 0.0174
轮次	6	训练:[300, 700)	验证:[700, 760)	样本: 60	MAE: 0.0084
轮次	7	训练:[360, 760)	验证:[760, 820)	样本: 60	MAE: 0.0138
轮次	8	训练:[420, 820)	验证:[820, 880)	样本: 60	MAE: 0.0113
轮次	9	训练:[480, 880)	验证:[880, 940)	样本: 60	MAE: 0.0111
轮次	10	训练:[540, 940)	验证:[940, 1000)	样本: 60	MAE: 0.0109
轮次	11	训练:[600, 1000)	验证:[1000, 1060)	样本: 60	MAE: 0.0180
轮次	12	训练:[660, 1060)	验证:[1060, 1120)	样本: 60	MAE: 0.0138
轮次	13	训练:[720, 1120)	验证:[1120, 1180)	样本: 60	MAE: 0.0221
轮次	14	训练:[780, 1180)	验证:[1180, 1240)	样本: 60	MAE: 0.0146
轮次	15	训练:[840, 1240)	验证:[1240, 1300)	样本: 60	MAE: 0.0086
轮次	16	训练:[900, 1300)	验证:[1300, 1360)	样本: 60	MAE: 0.0152
轮次	17	训练:[960, 1360)	验证:[1360, 1420)	样本: 60	MAE: 0.0117
轮次	18	训练:[1020, 1420)	验证:[1420, 1480)	样本: 60	MAE: 0.0102

滚动回测完成，共 18 轮

平均 MAE: 0.0168

最优 MAE: 0.0084 (第 6 轮)

最差 MAE: 0.0595 (第 1 轮)

2.

2.2. 训练和验证流程变动

与目标 2 的一次性全样本训练相比，滚动回测的主要流程变动如下：

- 数据划分方式变化：**目标 2 按固定比例一次性划分训练/测试集；目标 3 通过滑动窗口动态生成多组训练/验证数据，每轮数据区间不同。
- 模型训练方式变化：**目标 2 训练一个模型；目标 3 每轮独立训练一个新模型，共训练 18 个模型。
- 归一化方式变化：**每轮使用当前训练窗口数据拟合 Scaler，验证窗口使用同一 Scaler 转换，防止数据泄露。
- 评估方式变化：**目标 2 计算整体 MAE；目标 3 记录每轮 MAE 并汇总统计，评估模型在不同时间段的稳定性。


```

def rolling_backtest(prices, lookback=30, train_window_size=400,
                    step=60, device='cpu', epochs=25, batch_size=32):
    """
    滚动回溯：
    第r轮用 [r*step, r*step+train_window_size) 的数据训练
    在 [r*step+train_window_size, r*step+train_window_size+step) 上验证
    每轮使用训练窗口自己的scaler归一化，防止未来信息泄露
    """
    total = len(prices)
    n_rounds = (total - train_window_size - lookback - 1) // step
    print(f"总数据量: {total} | 训练窗口: {train_window_size} | 步长: {step}")
    print(f"预计轮次: {n_rounds}\n")

    results = []

    for r in range(n_rounds):
        train_start = r * step
        train_end = train_start + train_window_size
        val_end = train_end + step

        if val_end + 1 > total:
            break

        # 训练数据
        train_prices = prices[train_start: train_end + 1]
        scaler = MinMaxScaler(feature_range=(-1, 1))
        train_scaled = scaler.fit_transform(train_prices.reshape(-1,1)).flatten()
        X_train, y_train = make_regression_samples(train_scaled, lookback)

        if len(X_train) < 10:
            continue

        # 训练模型
        model = train_window(X_train, y_train, device, epochs, batch_size)

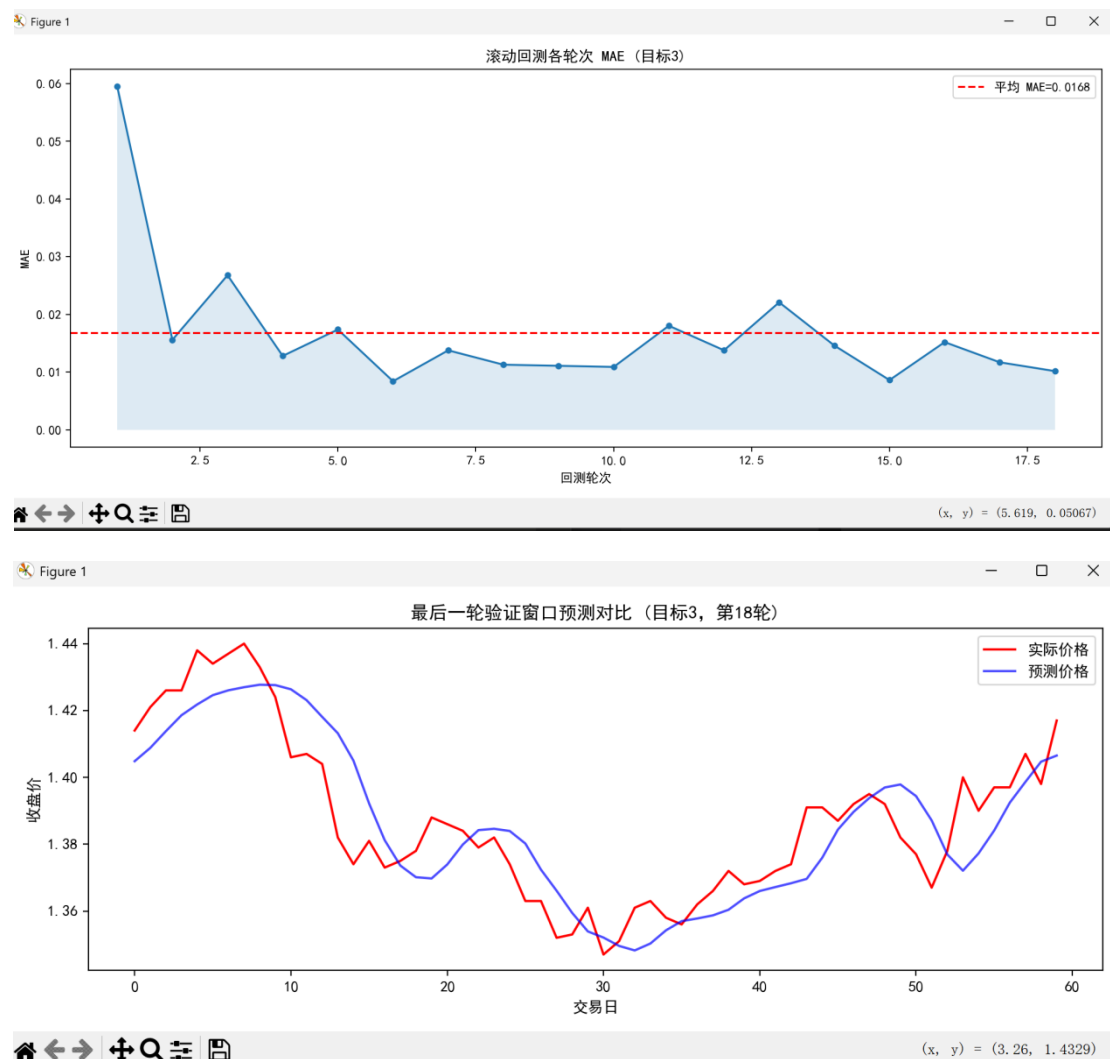
        # 验证数据 (用训练集的scaler, 防止未来泄露)
        val_ctx_start = train_end - lookback
        val_prices_ctx = prices[val_ctx_start: val_end + 1]
        val_scaled_ctx = scaler.transform(val_prices_ctx.reshape(-1,1)).flatten()
        X_val, y_val_scaled = make_regression_samples(val_scaled_ctx, lookback)

        if len(X_val) == 0:
            continue

        model.eval()
        with torch.no_grad():
            preds_scaled = model(torch.tensor(X_val).to(device)).cpu().numpy()

```

2.3. 训练和验证结果



滚动回测共完成 18 轮，平均 MAE 为 0.0168。各轮次 MAE 整体稳定，后期轮次（第 6 轮以后）MAE 基本维持在 0.01~0.02 之间，说明模型在不同时间窗口下均具有较好的预测稳定性。最后一轮验证窗口的预测曲线与真实价格走势基本吻合，验证了模型的有效性。